Сценарии интервью

Спонсор проекта со стороны бизнеса

Цель: зафиксировать бизнес-цели, приоритеты, ожидаемый эффект, допустимые ограничения, критерии успеха и риск-аппетит.

Предварительные гипотезы:

- Главный запрос — не «переписать», а повысить скорость и предсказуемость изменений.
- Критичны оформление заказа (checkout), оплата и обещание доставки.
- Полная одномоментная миграция неприемлема.
- Нужна промежуточная бизнес-ценность на каждом этапе.
- Важна единая картина данных для маркетинга, операций и финансов.
- Импортонезависимость важна, но не первоочередна
- Бизнес готов на параллельные инвестиции в стабилизацию и новые фичи

Вопросы:

- Какие бизнес-результаты должны быть через 3, 6, 12 месяцев?
- Как вы будете измерять успех программы?
- Как приоритизируете: скорость вывода новых фич, стабильность ядра, прозрачность данных?
- Какие новые сценарии (персонализация, партнёрские) хотите запустить первыми?
- Какие изменения в коммерческих сценариях сейчас самые болезненные?
- Что для вас абсолютно недопустимо при трансформации?
- Какой уровень деградации (ухудшения производительности, задержки отклика, тайм ауты и пр) допустим в пиковые периоды заказов?
- Какие компоненты/модули можно безопасно менять (перерстраивать) в первую очередь , а какие требуют тщательного планирования и тестирования изменений?
- Какой бюджетный и временной горизонт считается приемлемым?
- Кто принимает решение о приоритетах и компромиссах?
- Какие данные/отчёты сейчас критически не хватает бизнесу?

- **Критерии ценности ответов:**

- Есть измеримые KPI и приоритеты.
- Зафиксированы явные ограничения и зоны недопустимого риска.
- Понятны критерии приёмки этапов.
- Определены владельцы решений.

3.2. Руководитель команды заказов и оформления

Цель: восстановить фактический процесс оформления и обработки заказа, выявить исключения, ручные операции, узкие места и зависимости.

Предварительные гипотезы:

- Оформление заказа (checkout) — монолитная зона с исторической логикой, содержит тяжёлые синхронные вызовы, замедляющие пики.
- Много исключений: частичные отмены, раздельная доставка, повторные оплаты.

- Поддержка разных каналов продаж приводит к реализации собственных сценариев поведения.
- Цена, промо, доставка часто требуют ручных корректировок.
- Карточка заказа не является единым источником правды.

Вопросы:

- Проведите заказ от создания до завершения: где точки принятия решений?
- Какие внешние вызовы происходят синхронно?
- Что чаще всего ломается и как это обнаруживается?
- Какие ручные операции выполняются и кем?
- Как часто возникают исключения и на каких каналах?
- Какие данные нужны для разбора заказа?
- Какие статусы используются и где они противоречат друг другу?
- Какие пиковые нагрузки и как они влияют на процесс?
- Какие обходные схемы стали постоянными?
- Что происходит, если внешняя система не отвечает в процессе оформления?
- Как отслеживается путь заказа через компоненты? Есть трассировка?
- Какие типовые изменения занимают больше всего времени и почему?
- Что бы вы изменили в первую очередь, если бы могли без риска?
- Где логика промо/скидок — в монолите, в БД, в сервисах?

Критерии ценности ответов:

- Построена реальная карта процесса.
- Список исключений и ручных операций с частотой.
- Выявлены системные зависимости и узкие места.
- Сформулированы требования к статусам, истории и наблюдаемости.

3.3. Технический лидер / ведущий разработчик монолитного ядра

Цель: получить фактическое устройство системы, технические ограничения, зависимости, узкие места и технический долг.

Предварительные гипотезы:

- Java-монолит + Oracle — центральный узел.
- Взаимодействие через прямые вызовы, БД и async.
- Сервисы связаны с монолитом через staging-таблицы — скрытая связанность
- Границы модулей размыты, бизнес-логика распределена.
- Много временных решений, ставших продуктивными.
- Ограниченная виртуализация и ресурсы на пределе возможности.
- Статусная логика реализована в нескольких местах, модель статусов отличается.

Вопросы:

- Какие модули внутри монолита и где границы?
- Как происходит обмен между монолитом и сервисами на Python/.NET?
- Есть ли распределённая трассировка? Как ищут причины сбоев?
- Как реализованы транзакции и консистентность?
- Какие схемы БД используются как интеграционные?

- Где находятся главные узкие места (bottlenecks)?
- Какие зависимости от Oracle и зарубежных компонентов?
- Насколько реалистичен отказ от Oracle? Что для этого нужно?
- Какие зависимости между сервисами вы бы назвали критическими?
- Какие участки монолита можно выделить в сервисы безопасно и в первую очередь?
- Какие есть сложности (связи) для поэтапной декомпозиции?
- Как устроены мониторинг, логирование, трейсинг?
- Какой технический долг наиболее критичен?

Критерии ценности ответов:

- Проверена контекстная, компонентая схема.
- Есть матрица зависимостей.
- Список узких мест и рисков.
- Определены кандидаты на выделение.
- Зафиксированы технические ограничения и долг.